

AMBA APB Protocol: From Verilog Code to a Manufacturable Layout

A beginner's walkthrough of a complete open-source chip design flow

This report follows one small digital design all the way from a few hundred lines of Verilog to a layout file a semiconductor factory could manufacture. Every stage is explained from first principles, and every number quoted was produced by running the tool, not estimated.

STAGE	WHAT IT PRODUCED	RESULT
Simulation	34 of 34 self-checking tests	✓ PASS
Synthesis	1,732 Sky130 logic gates	✓ PASS
Routing	1,857 nets wired, 0 violations	✓ PASS
Timing	Meets 40 ns at both corners	✓ PASS
DRC and LVS	0 errors, circuits match	✓ PASS

CONTENTS

What is in this document

01	Start here	p. 03
02	Chip design in ten minutes	p. 04
03	What APB is, and why it exists	p. 05
04	Getting the RTL right	p. 06
05	Lint	p. 07
06	Simulation	p. 08
07	Verification in Python	p. 09
08	Synthesis	p. 10
09	Floorplan and placement	p. 11
10	Clock tree and routing	p. 12
11	Timing signoff	p. 13
12	GDS streamout	p. 14
13	DRC and LVS	p. 15
14	Everything at a glance	p. 16
15	Run it yourself	p. 17
16	Glossary	p. 19

SECTION 1

Start here

This document describes a real chip design, and it assumes you have never done one. If you can read a little code and know roughly what a logic gate is, that is enough. Every term is defined the first time it appears, and the last two pages are a glossary.

WHAT WE BUILT

APB_Protocol is a small digital circuit that moves data between one controller and two memory blocks, following a published industry standard called **AMBA APB**. It is about four hundred lines of **Verilog**, a language for describing hardware. Section 3 explains what the protocol does and why a chip needs one.

WHAT WE DID WITH IT

We took that Verilog through every step a real chip goes through, using only free and open tools, and stopped one step short of manufacturing. The end product is a **GDS** file: the format a semiconductor factory reads to make photomasks. Each stage was actually run, and every figure in this report is what the tool printed.

WHAT CAME OUT

QUESTION A REVIEWER ASKS	WHAT WE MEASURED	VERDICT
Does the logic behave correctly?	34 of 34 checks, three ways	✓ PASS
Can it be built from real gates?	1,732 Sky130 standard cells	✓ PASS
Can it be wired up?	1,857 nets, 0 violations	✓ PASS
Is it fast enough?	Meets 40 ns at both corners	✓ PASS
Is the layout legal?	0 DRC errors, LVS matches	✓ PASS

What this document does not claim

No wafer was manufactured and no silicon has been measured. The claim is that this design is ready to hand to a factory, not that it has been through one.

SECTION 2

Chip design in ten minutes

Turning a hardware description into something a factory can build is a pipeline. Each stage takes the previous stage's output, makes one specific decision, and hands on a more concrete version of the same circuit. Here is the whole path, and the folder in this project where each step lives.

THE PIPELINE, STAGE BY STAGE

FOLDER	WHAT HAPPENS HERE, IN PLAIN ENGLISH
00_lint	A spell-checker for hardware code. Flags risky patterns.
01_rtl	The Verilog source: what the circuit should do.
02_sim	Run the design on test inputs and check the outputs.
03_verification	A second, harder test suite written in Python.
04_synthesis	Turn the description into real logic gates.
05_dft	Test circuitry for the factory. Not used in this project.
06_floorplanning	Decide the chip's size and where power comes in.
07_placement	Give every gate a real position on the chip.
08_sta	Check the circuit is fast enough to run at speed.
09_place_and_route	Build the clock network, then draw every wire.
10_gds	Write the layout in the factory's file format.
11_physical_verification	Prove the layout is legal and matches the circuit.

You do not run these by hand, one at a time

Each folder has a `Makefile`, and the project root has one that calls them in order. `igny run make flow` runs the entire pipeline. Section 15 has the full command list.

SECTION 3

What APB is, and why it exists

APB stands for Advanced Peripheral Bus. It is part of AMBA, a family of connection standards published by Arm and used across most of the industry. It answers a question every chip has to answer: how do the parts talk to each other?

THE PROBLEM IT SOLVES

A chip has a few things that issue commands, such as a processor, and many things that respond to them: timers, serial ports, small memories. Wiring every pair directly does not scale. A **bus** is the shared alternative, one set of wires plus rules about who may drive them and when. APB is the simplest bus in the AMBA family, intended for the slow peripherals where a faster bus would be wasted.

HOW ONE TRANSFER WORKS

The design is a **finite state machine**: a circuit that is always in exactly one named state and moves between states on fixed rules. An APB transfer walks through three of them and takes at least two clock cycles.

IDLE is the resting state, with no transfer in progress.

SETUP lasts exactly one cycle. The controller puts the address and, for a write, the data onto the wires, and selects which memory it is talking to.

ACCESS is where the transfer completes. The controller confirms the transfer is live, and stays in this state until the responding device raises a ready signal. That signal is how a slow device asks for more time.

Why the state machine is worth this much attention

Four of the five defects in Section 4 are in these transitions. A bus protocol is a contract about exactly when each signal may change, so a state machine that is almost right produces a design that almost works, which is the hardest kind to debug.

SECTION 4

Getting the RTL right

RTL means register transfer level: hardware described as registers that hold values and the logic that moves values between them on each clock tick. Verilog is the language; the RTL is what you write in it. Before any physical work, the RTL has to be correct, because every later stage faithfully reproduces whatever it is given.

THE STARTING POINT

The design arrived hand-written: one requester called **master_bridge** and two responding memories, **slave1** and **slave2**. Read against the Arm specification (document IHI 0024D) alongside the code, it had five real defects. Each one is a place where the code and the published state diagram disagree.

THE FIVE DEFECTS

The state machine could skip a step. The move from SETUP to ACCESS was written as conditional. The specification makes it unconditional: once in SETUP, the next cycle is ACCESS, always.

It waited on the wrong signal. Leaving ACCESS was gated on a testbench-only signal instead of PREADY, the one signal the standard says controls that transition.

Two blocks raced. One piece of code wrote the next-state variable immediately while another read it on the same clock edge, so which won depended on the simulator's internal ordering rather than on the design.

The write/read flag could move mid-transfer. PWRITE was recalculated every cycle instead of captured once, so it could glitch part-way through a transfer the standard requires it to stay stable through.

Every read returned the previous read's data. An address register added earlier to fix a different problem put one extra cycle of delay on the read path only, so reads were permanently off by one transaction.

A sixth defect, in the test code itself

The original testbench never checked what came back; it only drove inputs. Rewritten to compare every read against a **shadow copy** of what memory should contain, it immediately found the fifth defect, which the old version structurally could not catch.

SECTION 5

Lint

Lint is the cheapest check available. The tool reads the code without running it and flags patterns that are legal but usually mistakes: a signal declared and never used, a value being silently truncated. It cannot prove a design correct. It catches careless errors before the slow tools spend an hour on them.

WHAT WE RAN

```
# Verilator reads the RTL without simulating it
igny run make lint
8 warnings, 0 errors
```

WHAT THE EIGHT WARNINGS ACTUALLY ARE

Two are a naming convention. Verilator expects a file's name to match the module inside it, and `master.v` contains a module called `master_bridge`. Cosmetic, and left alone on purpose.

Six trace to one root cause. The two memories accept an 8-bit address but hold only 64 entries, which needs 6 bits, so the top two bits go nowhere. Harmless for how the design is used today, and worth knowing for anyone who widens the address space later.

Warnings are not failures here

Lint is a review gate, not a pass/fail stage. Eight warnings with no real defect behind them is a reasonable place to stop. Suppressing them to reach a clean number would hide the address-width note, which is the one a future reader needs.

SECTION 6

Simulation

Simulation runs the design on a computer before any hardware exists. You write a second program, a **testbench**, that drives inputs, waits, and checks what comes back. A testbench that only drives inputs proves nothing. The value is entirely in the checking.

WHAT THE TEST DOES

- 1. Write.** Put 16 different values into each of the two memories, 32 writes in all.
- 2. Read back and check.** Read all 32 locations and compare every value against a **shadow model**, a plain software copy of what memory should contain. Any mismatch fails the test on the spot.
- 3. Stress the handover.** Run two writes back to back with the request held across the SETUP to ACCESS to SETUP boundary. This is the exact sequence that only works because of the state machine fix in Section 4.

RESULT

```
# compile the design and testbench, then run it
igny run make sim
==== PASS: all checks matched ====
34 / 34 checks passed, 0 mismatches
```

Three independent testbenches agree

This Verilog testbench, the Python smoke test and the full Python verification environment in Section 7 all report the same 34 of 34. Three separately written checkers converging on one answer is a stronger signal than any one of them passing alone.

SECTION 7

Verification in Python

A testbench written by hand tests what its author thought to test. **UVM**, the Universal Verification Methodology, is the industry's answer: a standard structure that separates generating stimulus, driving the wires, watching the wires, and judging the result, so each part can be reused and randomised.

THE SAME METHODOLOGY, WITHOUT THE LICENCE

UVM normally means SystemVerilog and a commercial simulator. Here it is **cocotb**, which lets Python drive an open simulator, and **pyuvvm**, which implements the UVM component structure in Python. Same architecture, free tools.

TWO LANES, ON PURPOSE

LANE	WHAT IT IS	RESULT
Smoke test	Minimal pyuvvm run, proves the plumbing works	34 / 34
Full environment	Driver, monitor, scoreboard and coverage	34 / 34 × 2

COVERAGE

Functional coverage answers a different question from whether the test passed: it asks which situations the tests actually reached. Direction, read or write, crossed with which memory was targeted, reached **100%** on all three measures.

Four defects found, all of them in the test code

Every problem this stage uncovered was in the verification code rather than the design: a configuration database wiped by the test runner, a monitor that assumed a signal stayed high, capture timing that differs between reads and writes, and a driver updating the bus in the wrong order. Verification code is code, and needs debugging too.

SECTION 8

Synthesis

Synthesis is where a description becomes a circuit. You wrote what the logic should do; the tool decides which physical gates implement it, choosing from a catalogue the factory supplies. That catalogue is the **standard-cell library**.

WHAT SKY130 IS

A **PDK**, or process design kit, is the bundle of files a factory publishes describing one manufacturing process. **Sky130** is SkyWater's open 130-nanometre PDK, and it is the reason a fully open flow like this one is possible at all. **Sky130 HD** is its high-density standard-cell library.

WHY THE SLOW CORNER

Silicon comes out slightly different every batch. Libraries are characterised at several **corners**, which are combinations of process quality, voltage and temperature. **SS** is the slow one. Mapping against SS means timing the design against the worst case a factory is allowed to ship, so faster batches are covered automatically.

RESULT

```
# map the RTL onto Sky130 gates, then regenerate the constraints
igny run make synth
Chip area for top module 'APB_Protocol': 39974.588800
Inferred latches: 0
```

Zero inferred latches took real work

A **latch** is memory the tool creates by accident when a piece of logic does not say what to do in every case. An earlier pass on this design produced 1,054 of them. They were removed by completing the logic properly, not by silencing the warning.

SECTION 9

Floorplan and placement

Until now the design has no physical form. The floorplan gives it one: how big the die is, where power enters, where the outside-world connections sit. Placement then decides where each of the 1,732 gates actually goes inside that outline.

FLOORPLAN

MEASURE	VALUE	WHAT IT MEANS
Die size	400 × 400 μm	The outline everything must fit inside
Standard cells	1,732	Individual logic gates to be placed
Placement rows	76	Horizontal tracks that cells snap onto
I/O pins	39	Connections to the world outside the chip

PLACEMENT

MEASURE	VALUE	WHAT IT MEANS
Utilization	31.2%	Share of the core area filled with cells
Legalization errors	0	No cell overlapping or off its row

The first die size was wrong, and the tool said so

A placeholder 250 × 250 μm die gave the placer less room than the netlist needs once spacing rules are applied: 115.8% utilization, which OpenROAD refuses outright. The die was resized to 400 × 400 μm using the figures OpenROAD printed in that failed run, and placement then converged.

SECTION 10

Clock tree and routing

Every register in the design is driven by the same clock, and the clock has to arrive everywhere at nearly the same instant. Clock tree synthesis builds a branching network of buffers to make that true. Routing then draws every other wire.

CLOCK TREE SYNTHESIS

One source cannot drive 88 registers directly; the signal would arrive late and distorted. **CTS** inserted **97 buffers** to fan it out. The spread between the earliest and latest arrival, called **skew**, came to 0.06 ns.

ROUTING

MEASURE	VALUE	WHAT IT MEANS
Nets routed	1,857	Separate electrical connections drawn
Total wire length	93,572 μm	About 9 cm of wire on a 0.4 mm die
Vias	17,887	Vertical links between metal layers
Router DRC errors	0	No shorts or spacing errors it can see

PARASITICS

Real wires have resistance and capacitance, and on a chip that **parasitic** load is a large share of the total delay. Extraction measures it from the finished routing and writes a **SPEF** file, which the timing check in Section 11 reads. Without it, timing would be an estimate rather than a measurement.

Two different rule checks, both called DRC

The zero above is the router checking its own work on the layers it drew. The foundry rule check in Section 13 is a separate and much stricter test against the factory's full rule deck. Both pass, and they are not the same claim.

SECTION 11

Timing signoff

A digital circuit works only if every signal arrives before the clock edge that captures it. **Static timing analysis** checks that for every path in the design at once, without simulating: it adds up gate and wire delays and compares them against the clock period.

THE TWO THINGS IT CHECKS

Setup asks whether a signal arrives early enough to be captured. **Hold** asks whether it stays stable long enough afterwards. The margin left over is called **slack**. Positive slack passes; negative slack is a failure.

THE TARGET

The clock period is **40 ns**, which is **25 MHz**. Every path has to complete inside that window at both process corners.

RESULT

CORNER	WORST SETUP SLACK	WNS	TNS
TT (typical)	+19.17 ns	0.00	0.00
SS (slow)	+4.18 ns	0.00	0.00

WNS is worst negative slack, the single tightest path in the design. **TNS** is total negative slack summed across every failing path. Both are zero, so nothing fails, and the slow corner still has 4.18 ns of margin left.



Timing closes at 40 ns on both corners

Real extracted parasitics, propagated clock, no timing repair required.

SECTION 12

GDS streamout

GDS is the file format a semiconductor factory reads. It contains polygons: the exact shapes that become each mask layer. Streamout is the conversion from the routed design into that format.

WHAT THE CONVERSION HAS TO GET RIGHT

Routing produces a **DEF** file, which records where each cell sits and where each wire runs but describes the cells themselves only as abstract outlines. The real shapes inside every gate live in the PDK's own GDS. The converter has to resolve all 19,745 placed cells to their true geometry rather than their outlines.

RESULT

```
# convert the routed design into the factory's format
igny run make gds
10_gds/APB_Protocol.gds  1.9 MB
19,745 cell instances written
```

CHECKED, NOT ASSUMED

An exit code of zero only means the tool did not crash. The GDS was dumped back to text and read directly: the top cell holds 19,745 real cell references, each carrying genuine mask polygons.

Why KLayout rather than Magic

The original plan used Magic for this conversion. KLayout's `strm2gds` resolves cell geometry straight from the LEF and DEF files with no Tcl dependency, which made it the simpler path. Magic is still used for the rule checking in Section 13.

SECTION 13

DRC and LVS

Two questions remain before a design can be handed over. Is the layout physically legal for this factory, and does the layout actually implement the circuit that was designed? **DRC** answers the first, **LVS** the second.

DRC - DESIGN RULE CHECK

Every process has geometric rules: minimum widths, minimum spacings, required enclosures. **Magic** loads the Sky130 rule deck and checks the streamed GDS against all of them. It reports **0 errors**.

LVS - LAYOUT VERSUS SCHEMATIC

Magic then extracts a circuit netlist back out of the layout polygons. **Netgen** compares that extracted circuit against the routed netlist the design tools produced. If a wire were missing, shorted or connected to the wrong place, the two would disagree.

VERIFIED COUNTS

MEASURE	FROM THE LAYOUT	FROM THE NETLIST
Devices	1,848	1,848
Nets	1,878	1,878



DRC and LVS both pass

0 Magic DRC errors. Netgen reports: Circuits match uniquely.

SECTION 14

Everything at a glance

Every stage, the tool that ran it, and the number that matters.

FULL FLOW RESULTS

STAGE	TOOL	KEY RESULT	STATUS
Lint	Verilator	8 warnings, 0 defects	✓ PASS
Simulation	Icarus Verilog	34 / 34 checks	✓ PASS
Verification	cocotb + pyuvvm	34 / 34, 100% coverage	✓ PASS
Synthesis	Yosys	1,732 cells, 0 latches	✓ PASS
Floorplan	OpenROAD	400 × 400 µm die	✓ PASS
Placement	OpenROAD	31.2% utilization	✓ PASS
Clock tree	OpenROAD	97 buffers, 0.06 ns skew	✓ PASS
Routing	OpenROAD	1,857 nets, 0 violations	✓ PASS
Timing	OpenSTA	+4.18 ns at slow corner	✓ PASS
GDS	KLayout	1.9 MB, 19,745 cells	✓ PASS
DRC	Magic	0 errors	✓ PASS
LVS	Netgen	Circuits match uniquely	✓ PASS

What this proves, and what it does not

Every stage ran, the layout is rule-clean, and it matches its own netlist. That is a complete digital implementation flow. It is not a manufactured chip: no wafer was run and no silicon has been measured.

SECTION 15

Run it yourself

Everything above is reproducible. The tools install through **igny**, Ignytion's environment manager, which keeps each project's toolchain isolated and versioned.

ONE-TIME SETUP

```
# create an environment and a workspace bound to it
igny env create apb_rtl_2_gds
igny workspace create --path ./apb_synthesis_with_sky130pdk --env apb_rtl_2_gds
cd apb_synthesis_with_sky130pdk
```

INSTALL THE TOOLS, THEN LOCK THE VERSIONS

```
igny tool install verilator
igny tool install iverilog
igny tool install yosys
igny tool install openroad
igny tool install magic
igny tool install netgen
igny tool install klayout-stream
igny workspace lock
```

RUN THE WHOLE FLOW

```
# every stage, in order, in one command
igny run make flow
```

OR ONE STAGE AT A TIME

```
igny run make lint
igny run make sim
igny run make verify
igny run make synth
igny run make floorplan
igny run make place
```

```
igny run make cts
igny run make route
igny run make sta
igny run make gds
igny run make physical
```

Where the detail lives

COMMANDS.md in the repository root lists every stage command with the source file to open alongside it. AGENTS.md carries the full prerequisite list and the debugging history behind each step.

SECTION 16

Glossary

Every term this document defines, in one place. Design and verification first, physical implementation on the next page.

DESIGN AND VERIFICATION

TERM	WHAT IT MEANS
AMBA	Arm's family of on-chip connection standards
APB	Advanced Peripheral Bus, the simplest AMBA bus
Bus	Shared wires plus rules about who may drive them
RTL	Hardware described as registers and the logic between them
Verilog	The language the RTL is written in
FSM	Finite state machine: logic in one named state at a time
Netlist	A list of gates and the wires connecting them
Lint	A check for risky patterns that does not run the code
Testbench	Code that drives a design and checks what comes back
Shadow model	A software copy of expected memory, used for checking
UVM	A standard structure for reusable verification code
cocotb	Library that lets Python drive a hardware simulator
pyuvvm	UVM component structure implemented in Python
Coverage	Which situations the tests actually reached

SECTION 16

Glossary, continued

PHYSICAL IMPLEMENTATION AND SIGNOFF

TERM	WHAT IT MEANS
Synthesis	Turning an RTL description into real logic gates
Standard cell	A pre-designed logic gate from the factory catalogue
PDK	Process design kit: the factory's files for one process
Sky130	SkyWater's open 130 nm process, used throughout here
Latch	Memory created by accident by incomplete logic
Corner	A process, voltage and temperature case to work at
Floorplan	The die outline, power grid and pin positions
Placement	Deciding the physical position of every gate
Utilization	Share of the core area occupied by cells
CTS	Clock tree synthesis: buffering the clock to arrive evenly
Skew	The spread in clock arrival times across the design
Routing	Drawing the metal wires between the placed gates
Via	A vertical connection between two metal layers
Parasitics	The resistance and capacitance of the wires themselves
SPEF	The file format that records extracted parasitics
STA	Static timing analysis: checks every path against the clock
Slack	Timing margin left over; negative slack is a failure
DEF	File describing where cells sit and where wires run
GDS	The polygon format a factory reads to make masks
DRC	Design rule check: is the layout physically legal?
LVS	Layout versus schematic: does the layout match the circuit?